

Code Explanation

GA Prompt Optimiser for LLM Code Generation

Evolving System Prompts on Codeforces Problems

Group 1 — Computational Intelligence, Spring 2026

April 2026

Condensed walkthrough: every file, every function, every call chain.

Backend: Google Cloud Vertex AI (Gemini 2.5 Flash & Pro).

Includes cross-file data-flow diagrams and complete worked examples.

Contents

1	Introduction & Architecture	2
1.1	What the Project Does	2
1.2	The Nine Files	2
1.3	Module Dependency Graph	2
1.4	Two-Model Architecture	3
1.5	Master Call-Flow (one generation)	3
2	config.py — Hyperparameters	5
2.1	Purpose & Design	5
2.2	Key Constants (current values)	5
3	seeds.py — Initial Population	7
4	dataset.py — Loading & Splitting Problems	8
4.1	Overview	8
4.2	load_problems() → list[dict]	8
4.3	split() → (evolution, holdout, reserved)	9
4.4	reserved_train_problems(problems, seed=42) → list[dict]	9
5	evaluator.py — C++ Extraction, Compilation & Testing	10
5.1	Overview	10
5.2	extract_code(response: str) → str	10
5.3	Process-Group Management: _kill_group()	10
5.4	_compile(code: str) → (bin_path None, error_msg)	11
5.5	run_tests_verbose() → dict	11
6	cost_tracker.py — Token & Cost Logging	13
6.1	Purpose & Design	13
6.2	CallRecord — One Call's Data	13
6.3	CostTracker — The Streaming Singleton	14
7	llm.py — All Vertex AI Calls	15
7.1	Overview	15
7.2	_get_api_client() — Lazy Singleton	15
7.3	_call_realtime_api() — The Core Wrapper	16
7.4	generate_code() — Fitness Evaluation	17
7.5	crossover() and the Three Mutation Operators	18
8	ga.py — The Genetic Algorithm	19
8.1	Logging Infrastructure	19
8.2	Individual — The Chromosome Dataclass	19
8.3	RNG State Serialisation	19
8.4	_eval_one_problem() and _eval_code_for_problem() — Per-Thread Workers . .	20
8.5	_assign_failure_examples(ind, collected_failures)	21
8.6	compute_fitness() → float	21
8.7	tournament_select, roulette_select, and breed	22
8.8	run() — The GA Loop	23

9	main.py — Orchestration & Entry Point	25
9.1	The Key Functions	25
9.1.1	Vertex AI Preflight (replaces <code>_check_cli()</code>)	25
9.1.2	<code>_find_latest_run(results_dir)</code> and <code>_load_resume_state()</code>	25
9.1.3	<code>save_checkpoint(run_dir, gen, population, stats, rng)</code>	25
9.1.4	<code>evaluate_on_holdout(best_prompt, holdout) → float</code>	26
9.1.5	<code>main()</code> — The 8-Step Pipeline	26
9.2	Output Files	27
10	genealogy.py — Ancestry Tracing	28
10.1	Purpose	28
10.2	The Three Core Functions	28
10.3	Genealogy Report Format	28
	Glossary	30

Chapter 1

Introduction & Architecture

1.1 What the Project Does

A **Genetic Algorithm (GA)** automatically discovers the best **system prompt** — a paragraph of English instructions — for a Gemini model that generates **C++ code**. Each prompt is a *chromosome*; fitness is the average fraction of test cases the LLM-generated C++ program passes on 65 Codeforces coding problems. Over 12 generations of selection, crossover, and mutation the GA evolves a prompt that solves more problems correctly.

Key idea

Manual prompt engineering is trial-and-error. This project automates it: the GA *searches* all possible English instruction paragraphs, using test-case pass rates as the fitness signal. The LLM never sees the test cases — only the problem statement.

1.2 The Nine Files

File	Lines	Role
config.py	83	Single source of truth for all hyperparameters and paths
seeds.py	137	15 hand-crafted generation-0 chromosomes (C++ focused)
dataset.py	253	Load 305 Codeforces problems; stratified split into evolution / holdout
evaluator.py	295	Extract C++ code; compile with g++ -std=c++23; run tests
cost_tracker.py	244	Stream token counts to JSONL; compute Vertex AI cost; write JSON reports
llm.py	470	All Vertex AI Gemini calls (code gen, crossover, mutation)
ga.py	717	The evolutionary loop: select, breed, evaluate (parallel), replace; 3 selection schemes
main.py	545	Orchestrate the run; checkpoint; resume; holdout; genealogy
genealogy.py	154	Trace ancestry of best evolved prompt through checkpoint files

1.3 Module Dependency Graph

```
main.py --imports--> config, dataset, evaluator, ga, llm
                      cost_tracker (singleton: tracker)
                      seeds         (SEED_PROMPTS list)
                      genealogy     (generate_report)
```

```

ga.py  --imports--> config, evaluator, llm

llm.py  --imports--> config
                        cost_tracker (singleton: tracker)
                        google.genai (Vertex AI SDK)

evaluator.py  ---> stdlib only  (os, re, signal, subprocess, tempfile, time)
dataset.py    ---> stdlib only  (json, random, sys, pathlib)
cost_tracker.py ---> stdlib only  (json, threading, time, dataclasses, datetime,
    pathlib)
genealogy.py  ---> stdlib only  (json, pathlib, collections)
seeds.py      ---> no imports

```

Network boundary

llm.py is the **only** file that touches the network. It uses the `google-genai` Python SDK with Google Cloud Vertex AI backend, authenticated via a service account JSON file (`service-account.json`). No interactive login or API key is required — the service account credential is loaded once at startup. All other files are pure computation or local disk I/O.

1.4 Two-Model Architecture

Role	Model	Temp.	Purpose
Target (subject)	gemini-2.5-flash	0.0 (enforced)	Generate C++ code; fitness eval
Optimizer (tool)	gemini-2.5-pro	0.7 (enforced)	Crossover and mutation operators

Temperature enforcement

Unlike the previous Claude CLI design (where temperature flags were unavailable), the Vertex AI SDK enforces both temperatures exactly via `GenerateContentConfig(temperature=...)`. `TARGET_TEMPERATURE=0.0` makes code generation deterministic; `OPTIMIZER_TEMPERATURE=0.7` injects creative variation into the genetic operators.

Thinking tokens

Gemini 2.5 models generate internal chain-of-thought (*thinking tokens*) before producing the visible response. These are billed at the same output rate and are tracked separately via `usage_metadata.thoughts_token_count`. Budgets: `thinking_budget=512` for fitness calls (Flash), `thinking_budget=2048` for crossover/mutation (Pro).

1.5 Master Call-Flow (one generation)

```

main.main()
| -- Vertex AI preflight: verify service-account.json exists + client connects
| -- dataset.load_problems()      -> list[dict]  (305 Codeforces problems)
| -- dataset.split(problems, 65, 200, 42)  -> (evolution[65], holdout[200])
| -- tracker.set_log_path(run_dir/"calls.jsonl")
| -- tracker.set_live_path(run_dir/"costs_live.json")
| -- tracker.set_costs_live_path(COSTS_DIR/"cost_<run_dir_name>.json")
| -- llm.set_error_log_path(run_dir/"llm_errors.log")
|
+-- ga.run(SEED_PROMPTS[:5], evolution_pool, run_dir, on_generation=
    save_checkpoint,
        selection_scheme=args.scheme, elitism_count=1 if scheme=="elitism"
        else None)

```

```

| [GEN 0] for each of 5 seeds (sequential):
|   save seed_NN_<uid>.json checkpoint first (crash recovery)
|   compute_fitness(gen=0, ind, problems)    [65 problems, PARALLEL_EVALS
|   =5]
|   ThreadPoolExecutor(max_workers=5):
|     _eval_one_problem(gen, ind, prob, idx, total)
|     time.sleep(random.uniform(0,1))    [stagger concurrent requests]
|     -> llm.generate_code(ind.prompt, prob["question"])
|         client.models.generate_content(...) [Gemini Flash]
|         -> tracker.record(phase, model, tokens, cost)
|     -> _eval_code_for_problem(gen, ind, prob, idx, total, response,
|         llm_time)
|         evaluator.extract_code(response) -> C++ code string
|         evaluator.run_tests_verbose(code, inputs, outputs, timeout
|         =2)
|         _compile(code)                -> g++ -O2 -std=c++23 [Popen+
|         setsid]
|         _run_one(bin, stdin, 2) -> actual stdout            [Popen+
|         setsid]
|         _outputs_match(actual, expected)
|         _log_evaluation(entry)        -> evaluations_genNN.jsonl
|         returns (score, failure_example_or_None)
|         _assign_failure_examples(ind, collected_failures) -> ind.
|         failure_examples
|         fitness = mean(scores)
|
| [GENS 1..12] for each generation:
|   _select(population, rng) x2 -> parents [tournament/roulette per --
|   scheme]
|   breed(p1, p2, rng, allow_mutation=True or False) -> offspring
|   llm.crossover(a, b)                                [80% chance]
|   llm.mutate_inject(p, failure_examples) [25%] (if allow_mutation)
|   llm.mutate_delete(p)                             [15%] (if allow_mutation)
|   llm.mutate_rephrase(p)                             [10%] (if allow_mutation)
|   Note: first CROSSOVER_ONLY_SLOTS=2 offspring skip mutations
|   compute_fitness(gen, offspring, problems) [parallel, 65 problems each]
|   save_checkpoint(run_dir, gen, pop, stats) -> generation_NN.json
|
| Convergence check each gen:
|   CV = std/mean; if CV < 0.01 for 3 consecutive gens -> early stop
|
| +-- returns (best_Individual, history[list of dicts])
|
| -- evaluate_on_holdout(best.prompt, holdout[200]) [ThreadPoolExecutor,
| parallel]
| -- genealogy.generate_report(run_dir, best.uid)    -> genealogy_report.txt
| +-- tracker.flush(extra)    -> Code/Costs/cost_TIMESTAMP.json

```

API call scale

One full run (no early stop) makes approximately:

$5 \times 65 = 325$ Flash calls (gen 0) + $12 \times 5 \times 65 = 3,900$ Flash calls (gens 1–12) + 200 Flash calls (holdout) = **4,425 Gemini Flash calls**.

Crossover/mutation: $\approx 12 \times (5 \times 0.8) + (3 \times 0.25) + (3 \times 0.15) + (3 \times 0.10) = 66$ Pro calls.

Estimated cost: $\approx \$2.12$ (Flash) + $\approx \$0.89$ (Pro) $\approx \$3.01$ total.

Chapter 2

config.py — Hyperparameters

2.1 Purpose & Design

Every number and path lives here. No other file contains hard-coded values. All constants are read at import time; changing `config.py` affects the whole system. Authentication uses a GCP service account JSON file — no API key or `.env` file is required.

2.2 Key Constants (current values)

Listing 2.1: `config.py` (complete)

```
1 from pathlib import Path
2
3 ROOT = Path(__file__).parent
4 APPS_TRAIN = ROOT / "Dataset" / "APPS" / "train"
5 APPS_TEST = ROOT / "Dataset" / "APPS" / "test"
6 RESULTS_DIR = Path(__file__).parent / "results"
7
8 # Dataset -- 305 pre-selected Codeforces problems
9 EVOLUTION_SIZE = 65 # stratified evolution pool (5 harder + 60 regular)
10 HOLDOUT_SIZE = 200 # held-out for final generalisation test
11 RANDOM_SEED = 42
12 EARLY_STOP_FAILURES = 3 # stop running tests after N consecutive failures
13
14 # Genetic Algorithm
15 POPULATION_SIZE = 5
16 GENERATIONS = 12
17 ELITISM_COUNT = 0 # no frozen elites -- all slots bred via crossover+
    mutation
18 CROSSOVER_ONLY_SLOTS = 2 # first N offspring per gen get crossover only (no
    mutation)
19 TOURNAMENT_SIZE = 3 # best-of-3 random draws for each parent
20
21 CROSSOVER_RATE = 0.8
22 MUT_INJECT_PROB = 0.25
23 MUT_DELETE_PROB = 0.15
24 MUT_REPHRASE_PROB = 0.10
25
26 # Google Cloud Vertex AI -- Gemini backend
27 SERVICE_ACCOUNT_PATH = Path(__file__).parent / "service-account.json"
28 GCP_REGION = "us-central1"
29
30 TARGET_MODEL = "gemini-2.5-flash" # code generation (fitness eval)
31 OPTIMIZER_MODEL = "gemini-2.5-pro" # crossover / mutation meta-prompts
32
33 TARGET_TEMPERATURE = 0.0 # enforced via GenerateContentConfig
34 OPTIMIZER_TEMPERATURE = 0.7 # enforced via GenerateContentConfig
35
36 API_FALLBACK_MAX_TOKENS_CODE = 8192 # max output tokens for code generation
37 API_FALLBACK_MAX_TOKENS_OPS = 700 # crossover / mutation outputs are short
38
39 # Convergence detection (CV = std/mean per generation)
40 CONVERGENCE_CV_THRESHOLD = 0.01 # 1% variation -> converged
41 CONVERGENCE_PATIENCE = 3 # consecutive stagnant generations to confirm
```

```
42 CONVERGENCE_MIN_MEAN      = 0.01    # only check CV when mean fitness > this
43
44 # Code execution (g++ compilation + binary runs)
45 EXEC_TIMEOUT              = 2        # seconds per test-case binary run
46 COMPILE_TIMEOUT           = 15       # seconds allowed for g++ compilation
47 PARALLEL_EVALS            = 5        # problems evaluated simultaneously per individual
```

Path change: ROOT is now Path(__file__).parent

ROOT previously resolved to the project parent directory (`parent.parent`). It now resolves to the Code/ directory itself (`parent`). Consequently the dataset path changed from `ROOT / "Dataset Examples" / ...` to `ROOT / "Dataset" / ...` to match the actual folder layout. This means all dataset files live under `Code/Dataset/APPS/{train,test}/`.

CROSSOVER_ONLY_SLOTS and elitism change

ELITISM_COUNT was lowered from 2 to 0 — no individuals are frozen across generations. Instead, CROSSOVER_ONLY_SLOTS=2 ensures the first 2 offspring per generation undergo only crossover (no mutation), providing stability without hard-locking elite individuals. The remaining 3 offspring per generation are eligible for all mutation operators.

Mutations are independent

Each of the three mutation probabilities is checked with a separate `rng.random()` call, so all three can fire on the same offspring (probability $0.25 \times 0.15 \times 0.10 = 0.00375$), or none can fire ($0.75 \times 0.85 \times 0.90 = 0.574$). Only offspring beyond CROSSOVER_ONLY_SLOTS are eligible for mutation.

Chapter 3

seeds.py — Initial Population

`seeds.py` defines `SEED_PROMPTS`: a `list[str]` of 15 diverse instruction paragraphs. All prompts require a **complete, compilable C++ program** as output; each emphasises a different cognitive strategy:

#	Strategy	Core emphasis
1	Structured analysis	Analyse input/output format, constraints, edge cases before coding
2	Terse and direct	Full C++ solution; compile with <code>g++ -std=c++23</code> ; just working code
3	Competitive programmer	Identify algorithm pattern (greedy, DP, graph, binary search, math)
4	Step-by-step reasoning	4-step: understand, identify algorithm, handle boundary, implement
5	Example-driven	Trace sample inputs manually; confirm behaviour before implementing
6	Conversational	Give me the full C++ source with main; handle all edge cases
7	Edge-case defender	$n=0$, $n=1$, all equal, maximum values, integer overflow
8	Algorithm-first	Determine algorithm + time/space complexity before writing code
9	Specification-heavy	5-point spec: cin/cout, <code>g++ -O2 -std=c++23</code> , all cases, exact format
10	Pseudocode-then-code	English sketch first, then translate to C++; cin/cout; include main()
11	Constraint-aware	If $n=10^5$, $O(n^2)$ will TLE; match complexity to constraints
12	Defensive / robust	Assume adversarial inputs at extreme constraint boundaries
13	Readability-first	Meaningful variable names; correct beats clever; standard I/O
14	Verify-against-examples	Mentally verify against sample inputs before writing code
15	Direct and focused	Entire C++ program; reads stdin; writes stdout; passes all tests

The file ends with `assert len(SEED_PROMPTS) >= 5` (previously `== 15`). The guard now allows populations smaller than 15 — `main.py` uses only `SEED_PROMPTS[:POPULATION_SIZE]` (the first 5 with `POPULATION_SIZE=5`). Seeds 2 and 9 reference `g++ -std=c++23` (updated from `c++17`).

In `ga.run()`, the selected seeds become `Individual` objects:

```
1 population = [Individual(prompt=p, operators="seed")
2               for p in seed_prompts[:config.POPULATION_SIZE]]
3 # Each Individual: prompt=<seed>, fitness=-1.0, uid=<random 8-char hex>,
4 #                   parent_a="", parent_b="", operators="seed",
5 #                   intermediate_steps={}, failure_examples=[]
```

Chapter 4

dataset.py — Loading & Splitting Problems

4.1 Overview

`dataset.py` loads 305 pre-selected Codeforces problems (stdin/stdout, C++ programs) and performs a **stratified split** into an evolution pool (65) and a held-out test set (200). Problem IDs are hard-coded in module-level dictionaries, ensuring reproducibility without requiring a specific dataset directory structure.

Why stratified?

A plain shuffle risks all harder problems landing in holdout. Stratified sampling draws a fixed count from each difficulty tier, guaranteeing the evolution pool sees problems at every rating level — including the harder 1800/1900-rated problems added in this version.

4.2 `load_problems()` → `list[dict]`

`load_problems()` → `list[dict]`

Called by: `main.main()`

Calls: `Path.exists()`, `json.loads()`, `Path.read_text()`

Iterates over the hard-coded `_SELECTED` dict (split → rating → folder IDs). For each problem folder it reads `question.txt` and `input_output.json`. Skips problems with missing files, bad JSON, or empty test cases.

Rating distribution (305 problems total):

Tier	Rating	Train	Test	Total
Regular	800	29	21	50
Regular	1000	10	40	50
Regular	1100	14	19	33
Regular	1200	23	10	33
Regular	1300	23	11	34
Regular	1400	27	5	32
Regular	1500	33	0	33
Regular	1600	17	0	17
Regular	1700	18	0	18
Harder	1800	3	0	3
Harder	1900	2	0	2
Total		199	106	305

Each passing problem produces:

```
1 {
2     "id":         "train/0002",    # "<split>/<folder>"
3     "question":  "Given n integers, ...",
```

```

4     "inputs":    ["3\n1 2 3\n", "1\n0\n"],      # raw stdin strings
5     "outputs":   ["6\n",          "0\n"],        # expected stdout strings
6     "rating":    800,
7 }

```

4.3 split() → (evolution, holdout, reserved)

```
split(problems, evolution_size=65, holdout_size=200, seed=42) -> (list, list, list)
```

Called by: main.main()

Calls: random.Random(seed), list comprehension, rng.shuffle(), rng.sample()

The stratified split algorithm:

```

split(problems, evolution_size=65, holdout_size=200, seed=42):

    rng = random.Random(seed)    # isolated, reproducible
    problems_by_rating = group problems by their "rating" field

    [HARDER PROBLEMS -- always go to evolution]
    harder_tiers = {1800, 1900}
    harder_problems = [p for p in problems if p["rating"] in harder_tiers]
    evolution = list(harder_problems)    # 5 problems (3+2)

    [REGULAR TIERS -- stratified draw]
    _EVOLUTION_PER_TIER = {800:7, 1000:6, 1100:6, 1200:7, 1300:7,
                          1400:7, 1500:7, 1600:6, 1700:7}    # sums to 60
    for each regular rating tier:
        tier_problems = [p for p in problems if p["rating"] == rating]
        rng.shuffle(tier_problems)    # seeded shuffle
        evolution += tier_problems[:count]    # take exactly count from this tier

    # Result: evolution has exactly 65 problems (5 harder + 60 regular)

    [HOLDOUT -- remaining problems, shuffled]
    remaining = [p for p in problems if p not in evolution set]
    rng.shuffle(remaining)
    holdout = remaining[:holdout_size]    # 200 problems
    reserved = remaining[holdout_size:]    # leftovers (small number)
    return (evolution, holdout, reserved)

```

4.4 reserved_train_problems(problems, seed=42) → list[dict]

```
reserved_train_problems(problems, seed=42) -> list[dict]
```

Called by: main.main() (optional, for analysis)

Returns: the 35 train-split problems not selected for evolution

This function returns the train-split problems that were shuffled out of the 65-problem evolution pool — useful for post-hoc analysis without touching the holdout set.

Fixed seed is critical

The same seed 42 must be used for all `split()` calls throughout a run. If the 65 evolution problems changed between generations, fitness scores from generation 3 and generation 7 would not be comparable. The seed is stored in `config.json` in each run directory.

Chapter 5

evaluator.py — C++ Extraction, Compilation & Testing

5.1 Overview

Two responsibilities: (1) strip C++ code out of an LLM’s raw text response, and (2) compile it with `g++` and run the resulting binary against test cases, returning a partial-credit score. Uses only the Python standard library, plus `signal` for process-group killing.

5.2 `extract_code(response: str) → str`

`extract_code(response) -> str`

Called by: `ga._eval_code_for_problem()`; `main.evaluate_on_holdout()`

Tries three strategies in order:

```
1 # 1. Tagged C++ fence:  '''cpp ... ''' or '''c++ ... '''
2 m = re.search(r"'''(?:cpp|c\+\+)\s*\n([\s\S]*)'", response, re.IGNORECASE)
3 if m: return m.group(1).strip()
4
5 # 2. Any fence:  ''' ... '''
6 m = re.search(r"'''\s*\n?([\s\S]*)'", response)
7 if m: return m.group(1).strip()
8
9 # 3. Assume raw code (no fences)
10 return response.strip()
```

5.3 Process-Group Management: `_kill_group()`

`_kill_group(proc) -> None`

Called by: `_compile()`, `_run_one()` on timeout

```
1 def _kill_group(proc):
2     try:
3         os.killpg(os.getpgid(proc.pid), signal.SIGKILL)
4     except ProcessLookupError:
5         pass
```

`SIGKILL` is sent to the entire **process group**, not just the parent. This prevents orphaned child processes (e.g., a spawned shell or forked worker) from persisting after a timeout. Both `_compile()` and `_run_one()` create new process groups via `preexec_fn=os.setsid` in `subprocess.Popen`.

5.4 `_compile(code: str) → (bin_path | None, error_msg)`

`_compile(code) → (str | None, str)`

Called by: `run_tests_verbose()`

Calls: `tempfile.mkstemp()`, `subprocess.Popen`, `proc.communicate(timeout=COMPILE_TIMEOUT)`

```
_compile(code):
    src_fd, src_path = tempfile.mkstemp(suffix=".cpp")    # temp .cpp file
    bin_path = src_path[:-4]                             # strip extension

    write code to src_path
    proc = subprocess.Popen(
        ["g++", "-O2", "-std=c++23", "-o", bin_path, src_path],
        stdout=subprocess.PIPE, stderr=subprocess.PIPE,
        preexec_fn=os.setsid)                            # new process group for kill_group()

    try:
        _, stderr = proc.communicate(timeout=COMPILE_TIMEOUT)
    except subprocess.TimeoutExpired:
        _kill_group(proc)
        proc.communicate()
        return None, "compile_timeout"

    if proc.returncode != 0:
        return None, stderr.decode()[:500]               # compile error
    return bin_path, ""                                  # success

    finally: os.unlink(src_path)                          # always delete .cpp
             if failed: os.unlink(bin_path)              # only delete binary on failure
```

`g++ -std=c++23 (not c++17)`

The compiler standard was updated from `-std=c++17` to `-std=c++23`. Seeds 2 and 9 in `seeds.py` were correspondingly updated to reference `c++23`. Modern Codeforces problems increasingly use C++23 features.

5.5 `run_tests_verbose() → dict`

`run_tests_verbose(code, inputs, outputs, timeout=2) → dict`

Called by: `ga._eval_code_for_problem()`

Calls: `_compile()`, `_run_one()`, `_outputs_match()`

```
run_tests_verbose(code, inputs, outputs, timeout):

    if not code or not code.strip():
        return {score:0.0, reason:"empty_code", ...}

    [COMPILE]
    bin_path, err = _compile(code)
    if bin_path is None:
        return {score:0.0, reason:"compile_fail", compile:{error:err}, ...}

    [RUN TESTS]
    passed = 0; consecutive_fails = 0

    for i in range(len(inputs)):
```

```

actual = _run_one(bin_path, inputs[i], timeout)
is_pass = actual is not None and _outputs_match(actual, outputs[i])

record test_cases[i] = {index, passed, actual, expected, time_s, timeout}

if is_pass: passed += 1; consecutive_fails = 0
else:
    consecutive_fails += 1
    if consecutive_fails >= EARLY_STOP_FAILURES:
        early_stopped = True; break

score = passed / len(inputs)      # denominator = TOTAL tests (fair penalty)
reason = "perfect" | "partial_pass" | "all_fail" | "all_fail_early_stop"

return {score, reason, compile, test_cases, tests_total, tests_run,
        early_stopped, total_test_time_s}

```

`run_tests()` is a thin wrapper returning only the float score:

```

1 def run_tests(code, inputs, outputs, timeout=2) -> float:
2     return run_tests_verbose(code, inputs, outputs, timeout)["score"]

```

Early stopping

After `EARLY_STOP_FAILURES=3` consecutive failures, the test loop exits. The score denominator is still `len(inputs)` (total), so skipped tests count as failures. This saves wall-clock time on clearly broken solutions.

Chapter 6

cost_tracker.py — Token & Cost Logging

6.1 Purpose & Design

Logs token counts and estimated costs for every Vertex AI call. The tracker streams each call record to a JSONL file immediately (no in-memory list), keeping memory usage flat regardless of run length. Three live JSON files are updated after every call; a final summary is written to `Code/Costs/` at run end.

Vertex AI billing

The project uses Google Cloud Vertex AI. Costs are real GCP charges — no flat subscription. The PRICING table contains verified Gemini rates (April 2026). Gemini 2.5 Flash: \$0.15/\$0.60 per MTok (in/out). Gemini 2.5 Pro: \$1.25/\$10.00 per MTok (in/out). The \$300 GCP free credit applies.

6.2 CallRecord — One Call's Data

```
1 PRICING = {      # (input_$/MTok, output_$/MTok)
2     "gemini-2.5-flash":      (0.15, 0.60),
3     "gemini-2.5-pro":       (1.25, 10.00),
4     "gemini-2.0-flash":     (0.10, 0.40),
5     "gemini-2.0-flash-001": (0.10, 0.40),
6     "claude-haiku-4-5@20251001": (1.00, 5.00), # kept for reference
7     "claude-haiku-4-5-20251001": (1.00, 5.00),
8     "claude-sonnet-4-6":    (3.00, 15.00),
9 }
10
11 @dataclass
12 class CallRecord:
13     phase: str; model: str
14     input_tokens: int; output_tokens: int
15     cost_usd: float | None = None # when set, used directly
16
17     @property
18     def total_cost(self) -> float:
19         if self.cost_usd is not None:
20             return self.cost_usd
21         rate_in, rate_out = PRICING.get(self.model, (0, 0))
22         return (self.input_tokens * rate_in
23             + self.output_tokens * rate_out) / 1_000_000
```

6.3 CostTracker — The Streaming Singleton

```
class CostTracker
```

Module-level singleton: `tracker = CostTracker()`

Imported by `llm.py` as: `from cost_tracker import tracker`

```
CostTracker.set_log_path(path)
    Called by: main.main() before GA starts
    Action: opens JSONL stream to calls.jsonl in the run directory

CostTracker.set_live_path(path)
    Called by: main.main()
    Action: path updated with current totals after every call (run_dir/costs_live.json)

CostTracker.set_costs_live_path(path)
    Called by: main.main()
    Action: second live path in Costs/ folder (Costs/run_live.json) updated every call

CostTracker.restore_from_jsonl(path) -> int
    Called by: main.main() on resume (--resume flag)
    Action: replays existing calls.jsonl to restore running totals; returns lines loaded

CostTracker.record(phase, model, input_tokens, output_tokens, cost_usd, extra)
    Called by: llm.py after every successful Vertex AI response
    Thread-safe (internal lock)
    Action:
        - updates running totals (_total_calls, _total_cost_usd, etc.)
        - writes one JSON line immediately to calls.jsonl (no in-memory list)
        {"t": "HH:MM:SS", "phase": ..., "model": ..., "in": ..., "out": ..., "cost": ..., "total_calls": ..., "running_cost": ...}
        - overwrites both live JSON files

CostTracker.flush(extra=None) -> Path
    Called by: main.main() at the very end
    Action:
        - assembles summary from running totals (_phase_totals, _model_totals)
        - writes Code/Costs/cost_TIMESTAMP.json
        - returns Path of the written file
```

Streaming vs. in-memory

Each record is written to disk immediately and discarded; running totals are maintained as simple scalars. A 4,425-call run uses negligible memory for cost tracking. The `extra` parameter to `record()` lets crossover/mutation calls embed input/output prompt text in the JSONL for audit purposes.

Chapter 7

llm.py — All Vertex AI Calls

7.1 Overview

The sole interface to Gemini. All public functions route through the private `_call_realtime_api()` wrapper, which uses the `google-genai` SDK to call Vertex AI, handles exponential-backoff retries, tracks thinking tokens, and logs costs.

```
Public API:
generate_code(system_prompt, question)          -> str    [C++ code text]
crossover(prompt_a, prompt_b)                   -> str    [merged prompt]
mutate_inject(prompt, failure_examples=None)     -> str    [prompt + new
    strategy]
mutate_delete(prompt)                           -> str    [prompt - weakest
    sentence]
mutate_rephrase(prompt)                         -> str    [rephrased prompt]
set_error_log_path(path)                       -> None   [llm_errors.log]

[Internal]
_get_api_client()                             -> google.genai.Client [lazy singleton]
_call_realtime_api(phase, model, system, user,
    max_tokens, retries=10) -> str
# temperature and thinking_budget derived from phase:
#   phase=="fitness" -> temperature=0.0, thinking_budget=512
#   all other phases -> temperature=0.7, thinking_budget=2048
_call(phase, model, system, user, api_max_tokens) -> str [thin wrapper]
```

7.2 `_get_api_client()` — Lazy Singleton

```
_get_api_client() -> google.genai.Client
```

Called by: `_call_realtime_api()` on first use

Thread-safe: double-checked locking with `_client_lock`

```
1 _api_client = None
2 _api_client_lock = threading.Lock()
3
4 def _get_api_client():
5     global _api_client
6     if _api_client is not None:
7         return _api_client          # fast path: already initialised
8     with _api_client_lock:
9         if _api_client is not None:
10            return _api_client       # double-check after acquiring lock
11        sa_info = json.loads(config.SERVICE_ACCOUNT_PATH.read_text())
12        project_id = sa_info["project_id"]
13        credentials = service_account.Credentials.from_service_account_info(
14            sa_info,
15            scopes=["https://www.googleapis.com/auth/cloud-platform"])
16        _api_client = genai.Client(
17            vertexai=True,
18            project=project_id,
19            location=config.GCP_REGION,
20            credentials=credentials,
```

```

21         http_options=HttpOptions(timeout=120_000), # 120 s per request
22     )
23     return _api_client

```

The singleton is initialised once and shared across all threads. No re-authentication happens during a run.

7.3 `_call_realtime_api()` — The Core Wrapper

```
_call_realtime_api(phase, model, system, user, max_tokens, retries=10) -> Optional[str]
```

Calls: `_get_api_client()`, `client.models.generate_content()`, `tracker.record()`, `time.sleep()`

Retries: 10 attempts, exponential backoff $5s \times 2^{\text{attempt}} + \text{jitter}$

Returns: response text; "" if all retries return empty; None if all retries fail

Temperature and thinking budget are derived internally from the `phase` parameter: `phase=="fitness"` $\Rightarrow$ `temperature=0.0`, `thinking_budget=512`; all other phases $\Rightarrow$ `temperature=0.7`, `thinking_budget=2048`.

Listing 7.1: `_call_realtime_api()` – Vertex AI invocation (abridged)

```

1  def _call_realtime_api(phase, model, system, user,
2      max_tokens, retries=10):
3      client = _get_api_client()
4
5      for attempt in range(retries):
6          t0 = time.monotonic()
7          try:
8              is_fitness = (phase == "fitness")
9              response = client.models.generate_content(
10                  model=model,
11                  contents=user,
12                  config=GenerateContentConfig(
13                      system_instruction=system,
14                      max_output_tokens=max_tokens,
15                      temperature=0.0 if is_fitness else 0.7,
16                      thinking_config=ThinkingConfig(
17                          thinking_budget=512 if is_fitness else 2048,
18                      ),
19                  ),
20              )
21              elapsed = round(time.monotonic() - t0, 2)
22              text = response.text or ""
23
24              # token accounting -- thinking tokens billed at output rate
25              usage = response.usage_metadata
26              in_toks = getattr(usage, "prompt_token_count", 0) or 0
27              out_toks = getattr(usage, "candidates_token_count", 0) or 0
28              think_toks = getattr(usage, "thoughts_token_count", 0) or 0
29              out_toks += think_toks # merge thinking into output
30
31              cost = _compute_gemini_cost(model, in_toks, out_toks)
32              tracker.record(phase=phase, model=model,
33                  input_tokens=in_toks, output_tokens=out_toks,
34                  cost_usd=cost)
35
36              if not text.strip(): # empty response -> retry
37                  wait = 5 * (2**attempt) + random.uniform(0, 3)
38                  time.sleep(wait); continue
39
40              return text # SUCCESS

```

```

41
42     except Exception as exc:
43         # classify: rate-limit (429), server error (500/503), timeout
44         # all retryable: wait = 5 * 2^attempt + jitter; time.sleep(wait)
45         wait = 5 * (2 ** attempt) + random.uniform(0, 3)
46         time.sleep(wait)
47
48     return None                                # all retries exhausted

```

What happens when `_call_realtime_api` raises `LLMCallError`

The caller (`generate_code` etc.) propagates the exception to `_eval_code_for_problem()`, which catches it and returns `(0.0, None)`. The problem scores 0; the run continues. One failed API call does not crash the GA.

7.4 `generate_code()` — Fitness Evaluation

`generate_code(system_prompt, question) -> str`

Called by: `ga._eval_code_for_problem()`; `main.evaluate_on_holdout()`

Calls: `_call_realtime_api(phase="fitness", model=TARGET_MODEL, thinking_budget=512, temperature=0.0)`

```

1  _CODE_GEN_DIRECTIVE = (
2      "\n\n"
3      "ABSOLUTE OUTPUT REQUIREMENT -- NO EXCEPTIONS:\n"
4      "Your entire response must consist of ONLY the raw C++ source code -- "
5      "one complete file, nothing else. Use C++23.\n"
6      "FORBIDDEN:\n"
7      " * Backtick code fences of any form (no '``', no ' ', no 'cpp' or 'c++' tags)\n"
8      " * Markdown formatting of any kind\n"
9      " * Any text before the code: no explanations, no preamble\n"
10     " * Any text after the code: no summaries, no notes\n"
11     "The FIRST character of your response must be the first character of the C++\n"
12     "code.\n"
13     "The LAST character must be the last character of the C++ code."
14 )
15 def _build_fitness_user_msg(question: str) -> str:
16     return (
17         f"{question}\n\n"
18         "Write a complete, compilable C++ program that reads from stdin and\n"
19         "writes "\n"
20         "the answer to stdout. Include main(). Compile target: g++ -O2 -std=c\n"
21         "++23.\n\n"
22         "DO NOT write any explanation, analysis, or reasoning.\n"
23         "DO NOT start with 'Looking at', 'I need to', or any English text.\n"
24         "Your response must begin with #include or int main -- nothing before it.\n"
25     )
26
27 def generate_code(system_prompt, question) -> str:
28     user_msg = _build_fitness_user_msg(question)
29     result = _call(
30         phase="fitness",
31         model=config.TARGET_MODEL,                                # gemini-2.5-flash
32         system=system_prompt + _CODE_GEN_DIRECTIVE,
33         user=user_msg,

```

```

32     api_max_tokens=config.API_FALLBACK_MAX_TOKENS_CODE, # 8192
33 )
34 if result is None:
35     raise LLMCallError(
36         f"generate_code: all retries failed (model={config.TARGET_MODEL})")
37 return result

```

7.5 crossover() and the Three Mutation Operators

```

crossover(prompt_a, prompt_b) -> str [phase="crossover", OPTIMIZER_MODEL,
    Pro]
    _call(phase="crossover", model=OPTIMIZER_MODEL,
        api_max_tokens=API_FALLBACK_MAX_TOKENS_OPS=700)
    -- temperature=0.7, thinking_budget=2048 derived from phase != "fitness"
    System: "You merge coding-instruction prompts. Output ONLY the merged prompt."
    User:   "Combine ... preserve strongest strategies ...
        -- Parent A -- {prompt_a}
        -- Parent B -- {prompt_b}"

mutate_inject(prompt, failure_examples=None) -> str [phase="mutate_inject", Pro]
    If failure_examples provided: appends up to 3 formatted failure entries
    (problem_id, rating, reason, compile_error/expected/actual) to guide the
    Optimizer.
    "Add one new, specific coding strategy ... (address at least one failure pattern
    )"
    Without failure_examples: generic inject -- "Add one new coding strategy ..."

mutate_delete(prompt) -> str [phase="mutate_delete", Pro]
    Edge case FIRST: sentences = re.split(r"(?<=[!?!])\s+", prompt)
    if len(sentences) <= 1: return prompt # single-sentence prompt; skip API call
    "Remove one sentence -- the one that contributes the least to code-generation
    quality."

mutate_rephrase(prompt) -> str [phase="mutate_rephrase", Pro]
    "Rephrase using different words, keep same meaning and all same strategies."
    Output: only the rephrased prompt text (no labels, no markdown).

```

failure_examples in mutate_inject

Each Individual stores up to 5 diverse failure examples (`individual.failure_examples`: a list of problem dicts with question, expected output, actual output). These are passed to `mutate_inject()` so the Optimizer can craft a strategy that specifically addresses the weaknesses the current prompt exhibits — targeted mutation rather than random injection.

Chapter 8

ga.py — The Genetic Algorithm

8.1 Logging Infrastructure

```
1 # Two log streams per run:
2 #   1. progress.log           -- human-readable timeline (thread-safe)
3 #   2. evaluations_genNN.jsonl -- one JSON line per problem evaluation
4
5 _progress_log: Path | None = None
6 _eval_log_dir: Path | None = None
7 _log_lock      = threading.Lock()
8 _eval_log_lock = threading.Lock()
9
10 def _log(msg, level="INFO"):      # timestamped, level-tagged; prints + appends
11 def _debug(msg):                  # shortcut for DEBUG-level entries
12 def _log_evaluation(entry):       # appends one JSON line to evaluations_genNN.
    jsonl
```

8.2 Individual — The Chromosome Dataclass

```
1 @dataclass
2 class Individual:
3     prompt: str          # the system prompt (the chromosome)
4     fitness: float = -1.0 # -1.0 = sentinel: not yet evaluated
5     uid: str = field(default_factory=lambda: uuid.uuid4().hex[:8])
6     parent_a: str = ""    # uid of first parent (empty for seeds)
7     parent_b: str = ""    # uid of second parent (empty for seeds)
8     operators: str = ""   # e.g. "crossover+inject" or "seed"
9     intermediate_steps: dict = field(default_factory=dict)
10    # intermediate_steps: per-problem score breakdown populated by
    compute_fitness
11    failure_examples: list = field(default_factory=list)
12    # failure_examples: up to 5 {question, expected, actual} dicts from failed
    tests
```

8.3 RNG State Serialisation

```
1 def rng_state_to_dict(state: tuple) -> dict:
2     """Serialise a raw MT19937 state tuple to a JSON-safe dict."""
3     version, internalstate, gauss_next = state
4     return {
5         "version": version,
6         "internalstate": list(internalstate), # tuple -> list for JSON
7         "gauss_next": gauss_next,
8     }
9
10 def rng_state_from_dict(d: dict) -> tuple:
11     """Restore raw state tuple from a dict saved by rng_state_to_dict."""
12     return (d["version"], tuple(d["internalstate"]), d["gauss_next"])
13
14 # Usage in run():
15 # Save: checkpoint["rng_state"] = rng_state_to_dict(rng.getstate())
```

```
16 # Restore: rng.setstate(rng_state_from_dict(checkpoint["rng_state"]))
```

These functions enable exact reproduction of any resumed run — the RNG state is saved in every `generation_NN.json` checkpoint so that generation N+1 draws the same random numbers whether resumed or run fresh.

8.4 `_eval_one_problem()` and `_eval_code_for_problem()` — Per-Thread Workers

Evaluation is split across two functions:

```
_eval_one_problem(gen, individual, prob, idx, total) -> (float, dict|None)
```

Called by: `compute_fitness()` via `ThreadPoolExecutor`

Calls: `llm.generate_code()`, then `_eval_code_for_problem()`

```
_eval_one_problem(gen, individual, prob, idx, total):
    time.sleep(random.uniform(0, 1)) # stagger concurrent API requests

    t_llm = time.monotonic()
    try:
        response = llm.generate_code(individual.prompt, prob["question"])
    except LLMCallError as exc:
        llm_time = round(monotonic() - t_llm, 3)
        return _eval_code_for_problem(gen, individual, prob, idx, total,
                                      None, llm_time) # None = infra failure

    llm_time = round(monotonic() - t_llm, 3)
    return _eval_code_for_problem(gen, individual, prob, idx, total,
                                response, llm_time)
```

```
_eval_code_for_problem(gen, individual, prob, idx, total, response_text,
llm_time_s) -> (float, dict|None)
```

Called by: `_eval_one_problem()`

Calls: `evaluator.extract_code()`, `evaluator.run_tests_verbose()`, `_log_evaluation()`

```
_eval_code_for_problem(gen, individual, prob, idx, total, response_text,
llm_time_s):
    if response_text is None: # infra failure path
        _log_evaluation({... , score:-1.0, reason:"infra_failure"})
        return (-1.0, None)

    code = evaluator.extract_code(response_text); del response_text

    if not code or not code.strip():
        test_result = {score:0.0, reason:"empty_response", ...}
    else:
        test_result = evaluator.run_tests_verbose(code, prob["inputs"],
                                                  prob["outputs"], EXEC_TIMEOUT)

    _log_evaluation({ts, generation, individual_uid, individual_operators,
                    system_prompt, problem_id, problem_rating, llm_call_time_s,
                    extracted_code, compile, test_cases, tests_total, tests_run,
                    early_stopped, total_test_time_s, score, score_reason})

    del entry, code

    # build failure_example if score < 1.0 (and not infra_failure)
    failure_example = {problem_id, rating, reason,
```

```

        expected[:200], actual[:200], compile_error[:200]}

    return (test_result["score"], failure_example)    # (float, dict|None)

```

8.5 _assign_failure_examples(ind, collected_failures)

_assign_failure_examples(ind, collected_failures) -> None

Called by: compute_fitness() after all threads complete

Modifies: ind.failure_examples (up to 5 diverse examples)

Selects up to 5 failure examples from collected_failures, prioritising diversity by failure reason using a priority ordering:

```

1 def _assign_failure_examples(individual, collected_failures):
2     _reason_priority = {
3         "compile_fail":      0,    # highest priority -- fix compilation first
4         "all_fail":          1,
5         "all_fail_early_stop": 1,
6         "partial_pass":      2,    # lowest priority
7     }
8     collected_failures.sort(
9         key=lambda f: _reason_priority.get(f["reason"], 3))
10    seen_reasons: set[str] = set()
11    diverse: list[dict] = []
12    for f in collected_failures:
13        if f["reason"] not in seen_reasons or len(diverse) < 3:
14            diverse.append(f)
15            seen_reasons.add(f["reason"])
16        if len(diverse) >= 5:
17            break
18    individual.failure_examples = diverse

```

Compile failures are prioritised highest — a prompt that produces uncompileable code should address that before worrying about partial passes. After sorting, each unique failure reason is admitted first; once diverse has 3+ entries, only new reasons are admitted until the cap of 5 is reached. These are stored on the Individual and passed to llm.mutate_inject() to target real weaknesses.

8.6 compute_fitness() → float

compute_fitness(gen, individual, problems) -> float in [0.0, 1.0]

Called by: run() – 5 times at gen 0; 5 times per generation (gens 1–12)

```

1 def compute_fitness(gen, individual, problems):
2     scores = [0.0] * len(problems)
3     collected_failures = []
4     _failures_lock = threading.Lock()
5
6     with ThreadPoolExecutor(max_workers=config.PARALLEL_EVALS) as executor:
7         futures = {
8             executor.submit(
9                 _eval_one_problem, gen, individual, prob, i, len(problems)
10            ): i
11            for i, prob in enumerate(problems)
12        }
13        for future in as_completed(futures):

```

```

14         idx = futures[future]
15         try:
16             score, failure_example = future.result()
17             scores[idx] = score
18             if failure_example is not None:
19                 with _failures_lock:
20                     collected_failures.append(failure_example)
21         except Exception as exc:
22             scores[idx] = -1.0
23
24     _assign_failure_examples(individual, collected_failures)
25     return sum(scores) / len(scores)

```

Expensive operation

One call to `compute_fitness()` makes **65 Gemini Flash API calls**. With `PARALLEL_EVALS=5`, these run in 5-wide batches. Evaluating all $5 \times 13 = 65$ new individuals in a full run (including gen 0 seeds) takes several hours.

8.7 tournament_select, roulette_select, and breed

```

tournament_select(population, rng):
    contestants = rng.sample(population, TOURNAMENT_SIZE=3)
    return max(contestants, key=lambda ind: ind.fitness)

roulette_select(population, rng):
    fitnesses = [max(ind.fitness, 0.0) for ind in population]
    total = sum(fitnesses)
    if total == 0:
        # all fitnesses zero: fall back to uniform
        return rng.choice(population)
    r = rng.uniform(0, total)
    cumulative = 0.0
    for ind, f in zip(population, fitnesses):
        cumulative += f
        if cumulative >= r:
            return ind
    return population[-1]
    # fallback if floating-point rounds short

breed(parent_a, parent_b, rng, allow_mutation=True):
    [BUILD COMBINED FAILURE LIST -- deduplicated union from both parents]
    combined_failures = []
    seen_ids = set()
    for f in (parent_a.failure_examples + parent_b.failure_examples):
        if f["problem_id"] not in seen_ids:
            combined_failures.append(f); seen_ids.add(f["problem_id"])
    combined_failures = combined_failures[:5]

    [CROSSOVER or COPY]
    if rng.random() < 0.80:
        prompt = llm.crossover(parent_a.prompt, parent_b.prompt); ops=["crossover"]
    else:
        prompt = rng.choice([parent_a.prompt, parent_b.prompt]); ops=["copy"]

    [MUTATIONS -- only if allow_mutation is True]
    if allow_mutation:
        if rng.random() < 0.25:
            prompt = llm.mutate_inject(prompt, combined_failures)
            ops += ["inject"]
        if rng.random() < 0.15:
            prompt = llm.mutate_delete(prompt); ops += ["delete"]

```

```

    if rng.random() < 0.10:
        prompt = llm.mutate_rephrase(prompt); ops += ["rephrase"]

    return Individual(prompt, parent_a=parent_a.uid, parent_b=parent_b.uid,
                      operators="+".join(ops))    # fitness=-1.0 (unevaluated)

```

8.8 run() — The GA Loop

```

run(seed_prompts, problems, run_dir, on_generation, resume_population,
    resume_start_gen, resume_history, resume_rng_state, selection_scheme="tournament",
    elitism_count=None) -> (best_Individual, history: list[dict])

```

Called by: main.main()

```

run(seed_prompts, problems, run_dir, on_generation,
    resume_population=None, resume_start_gen=0,
    resume_history=None, resume_rng_state=None,
    selection_scheme="tournament", elitism_count=None):

    _progress_log = run_dir / "progress.log"
    _eval_log_dir = run_dir
    history = resume_history or []

    [RESOLVE SELECTION SCHEME]
    _scheme = selection_scheme.lower()
    _select = roulette_select if _scheme == "roulette" else tournament_select
    _elitism = elitism_count if elitism_count is not None else config.ELITISM_COUNT

    rng = random.Random(config.RANDOM_SEED)    # seeded MT19937

    if resume_population:
        population = resume_population          # restore from checkpoint
        rng.setstate(rng_state_from_dict(resume_rng_state)) # exact MT19937 restore
        loop_start = resume_start_gen + 1
    else:
        population = [Individual(prompt=p, operators="seed")
                      for p in seed_prompts[:POPULATION_SIZE]]
        # Per-seed crash recovery: reload any seed_NN*.json checkpoints
        # from a previous interrupted gen-0 evaluation
        for i, ind in enumerate(population):
            if <seed checkpoint exists for seed i>:
                restore ind.uid, ind.fitness, ind.failure_examples from ckpt
            else:
                ind.fitness = compute_fitness(0, ind, problems)
                write seed checkpoint: {seed_idx, uid, fitness, failure_examples}
        loop_start = 1

    [GENS loop_start..GENERATIONS]
    stagnation_count = 0
    for gen in range(loop_start, GENERATIONS + 1):
        population.sort(by fitness, descending)
        stats = {generation:gen-1, best, mean, worst, std, cv, best_uid, best_prompt}
        history.append(stats)

    [CONVERGENCE CHECK]
    cv = std / mean (if mean > CONVERGENCE_MIN_MEAN, else -1.0)
    if cv >= 0 and cv < CONVERGENCE_CV_THRESHOLD:
        stagnation_count += 1
        if stagnation_count >= CONVERGENCE_PATIENCE:

```

```

        break    # early stop
    else:
        stagnation_count = 0

    if on_generation:
        on_generation(gen-1, population, stats, rng.getstate()) # saves
            generation_NN.json

    [ELITISM -- copy top _elitism individuals unchanged]
    new_pop = population[:_elitism] # typically 0 (tournament/roulette) or 1 (
        elitism scheme)

    [BREED n_offspring = POPULATION_SIZE - _elitism offspring]
    offspring_idx = 0
    while len(new_pop) < POPULATION_SIZE:
        p1 = _select(population, rng) # tournament or roulette per --scheme
        p2 = _select(population, rng)
        allow_mut = (offspring_idx >= CROSSOVER_ONLY_SLOTS) # first 2:
            crossover only
        child = breed(p1, p2, rng, allow_mutation=allow_mut)
        new_pop.append(child); offspring_idx += 1

    [EVALUATE offspring only (elites already have valid fitness)]
    for ind in new_pop[_elitism:]:
        ind.fitness = compute_fitness(gen, ind, problems)

    population = new_pop

    [FINAL] population.sort()
    history.append({generation:last_completed_gen, ...})
    return (population[0], history)

```

Three selection schemes (-scheme argument)

tournament (default): sample 3 individuals at random, pick the fittest. Balances exploitation with exploration; independent of absolute fitness values.

roulette: fitness-proportionate selection; probability of picking an individual is proportional to its fitness. Falls back to uniform random when all fitnesses are zero.

elitism: tournament selection + 1 elite individual copied unchanged each generation (**elitism_count**=1). Guarantees the best-ever individual survives.

Convergence detection (CV)

The coefficient of variation $CV = \sigma/\mu$ measures population homogeneity. When $CV < 0.01$ (less than 1% variation) for 3 consecutive generations and $\mu > 0.01$ (not a trivial all-fail population), the GA stops early. This prevents wasting API budget on a converged population.

Per-seed checkpoints for crash recovery

Each seed's fitness is saved to `seed_NN_<uid>.json` before moving to the next seed. If the run crashes midway through gen-0 evaluation, it re-loads completed seeds from disk rather than re-evaluating them.

Chapter 9

main.py — Orchestration & Entry Point

9.1 The Key Functions

9.1.1 Vertex AI Preflight (replaces `_check_cli()`)

Vertex AI credential check in `main()`

Called at startup, before dataset loading

Calls: `Path.exists()`, `_get_api_client()` (via `llm` module import)

Instead of checking for the `claude` CLI executable, `main()` verifies that `config.SERVICE_ACCOUNT_PATH` (`service-account.json`) exists on disk. It then imports `llm`, which triggers `_get_api_client()` — if the service account is invalid or the project lacks Vertex AI permissions, the error surfaces immediately before any dataset work.

9.1.2 `_find_latest_run(results_dir)` and `_load_resume_state()`

`_find_latest_run(results_dir) -> Path | None`

Called by: `main()` when `-resume` flag is given without an explicit path

Returns: the most recently modified run directory under `results/`

`_load_resume_state(run_dir) -> dict`

Called by: `main()` after finding the run to resume

Returns: `{population, start_gen, history, rng_state}` loaded from the latest `generation_NN.json` checkpoint

Resume support allows a crashed run to continue from the last saved generation, replaying `calls.jsonl` into `tracker.restore_from_jsonl()` to restore accurate running cost totals.

9.1.3 `save_checkpoint(run_dir, gen, population, stats, rng)`

`save_checkpoint(...)`

Called by: the `on_generation` lambda in `main()`

Output: `results/run_*/generation_NN.json`

Saves the full population state (all 5 prompts, fitnesses, parent UIDs, operators, `failure_examples`, `intermediate_steps`) plus the RNG state after each generation. The RNG state (MT19937 internals) is serialised via `rng_state_to_dict()` so that a resumed run produces identical subsequent draws.

9.1.4 `evaluate_on_holdout(best_prompt, holdout) → float`

```
evaluate_on_holdout(best_prompt, holdout, run_dir) -> float
```

Called by: `main()`

Calls: `ThreadPoolExecutor`, `llm.generate_code()`, `evaluator.extract_code()`, `evaluator.run_tests_verbose()`; writes `holdout_evaluations.jsonl`

Now runs **in parallel** using `ThreadPoolExecutor(max_workers=PARALLEL_EVALS)` (was sequential in the previous version). Evaluates all 200 held-out problems with the best evolved prompt; returns the mean partial-credit score.

9.1.5 `main()` — The 8-Step Pipeline

```
main():

[0] Parse CLI args
    --resume [RUN_DIR]                (optional: resume from crash)
    --scheme {tournament|roulette|elitism} (default: "tournament")
    --help

[1] Pre-flight checks
    verify SERVICE_ACCOUNT_PATH exists
    verify APPS_TRAIN / APPS_TEST paths exist (or warn)
    verify Vertex AI connectivity (lazy via llm module init)

[2] Load and split dataset
    all_problems = dataset.load_problems()                # 305 Codeforces problems
    evolution_pool, holdout, _ = dataset.split(
        all_problems, evolution_size=65,
        holdout_size=200, seed=42)

[3] Create timestamped run directory (or locate existing for resume)
    run_dir = RESULTS_DIR / "run_YYYYMMDD_HHMMSS_<scheme>"
           e.g. "run_20240501_143022_tournament"
    Writes: config.json          (hyperparameter snapshot)
           evolution_ids.json    (which 65 problem IDs)

[4] Initialise logging and cost tracking
    tracker.set_log_path(run_dir / "calls.jsonl")
    tracker.set_live_path(run_dir / "costs_live.json")
    tracker.set_costs_live_path(COSTS_DIR / f"cost_{run_dir.name}.json")
    llm.set_error_log_path(run_dir / "llm_errors.log")
    if resuming: tracker.restore_from_jsonl(run_dir / "calls.jsonl")

[5] Run GA
    resume_state = _load_resume_state(run_dir) if resuming else {}
    best, history = ga.run(
        seed_prompts=SEED_PROMPTS[:POPULATION_SIZE],    # first 5
        problems=evolution_pool,
        run_dir=run_dir,
        on_generation=_on_generation,                    # saves generation_NN.json + logs cost
        resume_population= resume_state["population"] if resume_state else None
        ,
        resume_start_gen= resume_state["start_gen"]      if resume_state else 0,
        resume_history=   resume_state["history"]        if resume_state else None
        ,
        resume_rng_state= resume_state["rng_state"]      if resume_state else None
        ,
        selection_scheme=args.scheme,
        elitism_count=1 if args.scheme == "elitism" else None)
```

```

[6] Save results
(run_dir / "history.json").write_text(json.dumps(history))
(run_dir / "best_prompt.txt").write_text(best.prompt)

[7] Holdout evaluation (parallel)
holdout_fitness = evaluate_on_holdout(best.prompt, holdout)
(run_dir / "summary.json").write_text({
    elapsed_minutes, best_prompt, best_fitness_evolution,
    best_fitness_holdout, best_uid, total_generations,
    resumed, selection_scheme})

[8] Genealogy report + cost flush
genealogy.generate_report(run_dir, best.uid)
-> run_dir/genealogy_report.txt, run_dir/genealogy_tree.json
tracker.flush(extra={...}) -> Code/Costs/cost_TIMESTAMP.json

```

9.2 Output Files

File	Contents
run_*/config.json	Exact hyperparameters (reproducibility)
run_*/evolution_ids.json	The 65 problem IDs used as the fitness pool
run_*/calls.jsonl	Per-call token + cost log (streaming JSONL)
run_*/costs_live.json	Live running-total cost summary
run_*/llm_errors.log	API retry / error events
run_*/progress.log	Human-readable timestamped timeline
run_*/evaluations_genNN.jsonl	Per-problem full evaluation records
run_*/seed_NN_<uid>.json	Per-seed gen-0 checkpoint (crash recovery)
run_*/generation_NN.json	Full population + RNG state per generation
run_*/history.json	Best / mean / worst fitness per generation
run_*/best_prompt.txt	The evolved prompt (main artefact)
run_*/holdout_evaluations.jsonl	Per-problem results for the 200 holdout problems
run_*/summary.json	Final fitnesses (evolution + holdout), scheme, runtime
run_*/genealogy_report.txt	Human-readable prompt ancestry chain
run_*/genealogy_tree.json	Full ancestry tree in JSON
Costs/cost_*.json	Final token counts and estimated cost breakdown per run
Costs/cost_<run_name>.json	Live cost summary updated after every API call (named after run dir)

Chapter 10

genealogy.py — Ancestry Tracing

10.1 Purpose

After a GA run completes, `genealogy.py` traces the lineage of the best evolved prompt back through all checkpoints to its seed ancestor. The output is both a human-readable report and a JSON tree.

10.2 The Three Core Functions

```
_load_all_individuals(run_dir) -> dict[uid, dict]
  Scans run_dir for all generation_NN.json checkpoint files.
  Builds a registry: {uid: {prompt, fitness, parent_a, parent_b,
                           operators, generation}}.
  Later generations overwrite earlier entries for the same uid (fitness may
  have been updated by re-evaluation on resume).

_collect_lineage(uid, registry) -> list[dict]
  Walks the parent_a chain from the best individual back to a seed.
  A seed has parent_a == "" (empty string).
  Returns a list of individual dicts ordered from seed to best.

_collect_full_tree(uid, registry, depth=0) -> dict
  Recursively collects both parents (parent_a and parent_b) at each node.
  Depth-limited to 40 to prevent infinite loops from any cycle in the graph.
  Returns a nested dict suitable for JSON serialisation.

generate_report(run_dir, best_uid) -> None
  Entry point called by main.main().
  1. _load_all_individuals(run_dir)
  2. _collect_lineage(best_uid, registry)    -> linear chain
  3. _collect_full_tree(best_uid, registry)  -> full ancestry tree
  4. Writes run_dir/genealogy_report.txt (human-readable text)
  5. Writes run_dir/genealogy_tree.json (full tree dict)
```

10.3 Genealogy Report Format

```
=== Genealogy Report ===
Best individual: uid=a3f2c1b9  fitness=0.7231  generation=11

Linear ancestry (seed -> best):
  Gen 0  uid=seed_1    fitness=0.4102  operators=seed
         prompt: "Solve the following C++ competitive programming..."
  Gen 3  uid=b2d44f1a fitness=0.5810  operators=crossover+inject
         parents: [seed_1 x seed_3]
         prompt: "Analyse the problem carefully. Consider time..."
  ...
  Gen 11 uid=a3f2c1b9 fitness=0.7231  operators=crossover+inject
         parents: [e1f2a3b4 x c5d6e7f8]
         prompt: "You are a competitive programmer. First identify..."

Full ancestry tree written to: run_dir/genealogy_tree.json
```

Why genealogy matters

The linear ancestry chain shows which genetic operations were most productive. If the best prompt descends through 3 consecutive **crossover+inject** operations, that suggests inject mutations are the key driver. If the chain is mostly **crossover+copy**, injection may not be contributing. This guides future hyperparameter choices.

Glossary

Chromosome	A candidate solution in a GA. Here: a system prompt string, stored as an <code>Individual</code> object.
Convergence detection	Stopping the GA early when the population is statistically homogeneous: $CV = \sigma/\mu < 0.01$ for 3 consecutive generations and $\mu > 0.01$. Prevents wasting API budget on a converged population.
CROSSOVER_ONLY_SLOTS	The first $N = 2$ offspring bred per generation receive crossover only (no mutation), providing stability without hard-locking elite individuals. The remaining 3 offspring per generation are mutation-eligible.
Crossover	Genetic operator combining two parents into one offspring. Here: the <code>Optimizer</code> LLM semantically blends two parent prompts. Fires with probability 0.8.
CV (coefficient of variation)	σ/μ of fitness across the population. Used by convergence detection.
Early stopping (test loop)	In <code>run_tests_verbose()</code> : halting the test loop after <code>EARLY_STOP_FAILURES=3</code> consecutive failures. The score denominator is still the total test count, so skipped tests count as failures.
Elitism	Copying the top k individuals unchanged into the next generation. <code>ELITISM_COUNT=0</code> in the current version — no frozen elites.
Evolution pool	The fixed 65 Codeforces problems used as the fitness benchmark. Chosen by stratified shuffle with seed 42 (5 harder problems always included).
Failure examples	Up to 5 problem failures stored on each <code>Individual</code> (<code>individual.failure_examples</code>). Passed to <code>mutate_inject()</code> so the <code>Optimizer</code> crafts strategies that address real weaknesses.
Fitness	Mean partial-credit score across 65 problems. Range: $[0.0, 1.0]$. Computed by <code>ga.compute_fitness()</code> .
Genealogy	The ancestry chain from the best evolved prompt back to its seed. Traced by <code>genealogy.py</code> after a run completes.
Gemini Flash	<code>gemini-2.5-flash</code> — the Target model used for code generation (fitness evaluation). Fast, cheap.
Gemini Pro	<code>gemini-2.5-pro</code> — the <code>Optimizer</code> model used for crossover and mutation meta-prompts. More capable, used for creative prompt manipulation.
Held-out set	200 problems never seen during evolution. Used to test generalisation of the final evolved prompt.
Individual	One GA population member: a prompt with fitness, uid, parent uids, operator history, intermediate steps, and failure examples.
Intermediate steps	Per-problem scores stored in <code>individual.intermediate_steps</code> as a dict mapping problem index to score. Used for analysis and checkpoint saves.

Meta-prompt	A prompt sent to the Optimizer LLM instructing it to modify another prompt (used in crossover and all three mutation operators).
Mutation	Modifying a single chromosome. Three independent types: inject (25%), delete (15%), rephrase (10%). Only applies to offspring beyond CROSSOVER_ONLY_SLOTS.
PARALLEL_EVALS	Number of problems evaluated simultaneously per individual (PARALLEL_EVALS=5, using a <code>ThreadPoolExecutor</code>).
Partial credit	Score = (tests passed) / (total tests). Provides a smooth gradient vs. binary pass/fail.
Process group	A set of processes sharing a PGID. <code>evaluator.py</code> creates new process groups via <code>preexec_fn=os.setsid</code> and kills them with <code>os.killpg(pgid, SIGKILL)</code> to prevent orphaned children after timeouts.
Resume	The <code>-resume</code> CLI flag re-loads the latest <code>generation_NN.json</code> checkpoint and continues the GA from the next generation, replaying cost totals from <code>calls.jsonl</code> .
RNG state	The internal state of Python's MT19937 Mersenne Twister random number generator. Serialised to JSON via <code>rng_state_to_dict()</code> and saved in every <code>generation_NN.json</code> checkpoint, enabling exact reproduction of resumed runs.
Service account	A GCP JSON credential file (<code>service-account.json</code>) used to authenticate Vertex AI API calls. No interactive login or API key required.
Singleton	The <code>tracker = CostTracker()</code> instance at module level is shared by all importers (<code>llm.py</code> and <code>main.py</code>).
Startup jitter	A random 0–2 s sleep at the start of each <code>_eval_code_for_problem()</code> call to stagger parallel API launches.
Stratified split	Sampling a fixed count from each difficulty tier (rating) when building the evolution pool, ensuring representation of all difficulty levels.
System prompt	English instructions sent before the user message, shaping LLM behaviour. The GA chromosome.
Temperature	LLM randomness parameter. Now enforced via <code>GenerateContentConfig(temperature=...)</code> : 0.0 for Flash (deterministic code gen), 0.7 for Pro (creative operator mutations).
Thinking tokens	Internal chain-of-thought tokens produced by Gemini 2.5 models before the visible response. Billed at the output token rate. Tracked via <code>usage_metadata.thoughts_token_count</code> . Budget: 512 for Flash, 2048 for Pro.
Tournament selection	Sample $k = 3$ individuals at random; return the fittest. Balances exploitation (picking the best) with exploration (not always the global best).
Two-model architecture	Gemini 2.5 Flash (code generation, target) is optimised; Gemini 2.5 Pro (reasoning, creative) performs the genetic operators.
Vertex AI	Google Cloud's managed ML platform. The project uses it to call Gemini models via the <code>google-genai</code> Python SDK with service account auth.